NPLify · P0 Technical Baseline

Architecture & Stack Note

New XP Technologies Limited

11 October 2026 · Confidential

1 · In one paragraph

NPLify is an internal tool. It runs as **one application** — screens, API and background jobs in a single Next.js / TypeScript service — against **one PostgreSQL database** whose ledger and audit tables accept inserts only, with **one S3 bucket** for slips, screenshots and photos, all inside **NPL's own AWS account in Hong Kong**. Staff sign in with NPLify's own login: email, password and a code from an authenticator app; users, roles and rights are rows in NPLify. Four outside services are used and none is trusted with the books: Kraken and CoinMarketCap for market rates, Tronscan and Etherscan to check transaction hashes, and one Google Sheet per client as that client's status journal. Daily backups are kept thirty days with restore to any minute. The running cost is about USD 50–80 a month. At ten deals a weekday the whole model is a few thousand rows a month; nothing here is sized for more than that, and nothing needs to be.

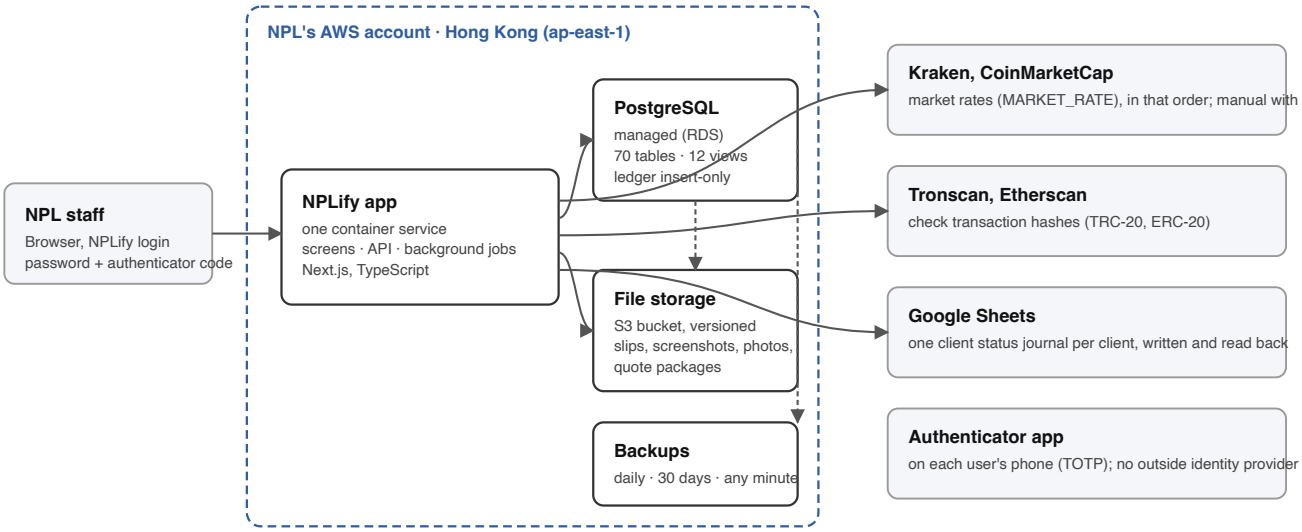


Figure 1 — How it fits together. One app, one database, one bucket inside NPL's account; four outside services, none of which holds NPL's data except the client's own journal sheet.

2 · Stack

Piece	NPL's page	What it has to do for the model
App	Next.js with TypeScript: screens, API and background jobs (quote clocks, rate fetch, alerts) in one app.	Screens per role (Section 5); a JSON API the screens call, which is also where every rule of the model is enforced — pricing (Calculation Specification), the state machines (Lifecycle & Approvals), maker-checker from ROLE_RIGHT rows, the field filter of ROLE.sees_economics; and a job runner inside the same service for the nine jobs of Section 4. One process, one deployment, one log.
Database	PostgreSQL. Ledger and audit tables accept inserts only; the database refuses updates and deletes.	70 tables and 12 derived views of the ERD. LEDGER_TRANSACTION, LEDGER_POSTING, LEDGER_ACCOUNT, AUDIT_LOG are insert-only at the database level (Section 3). Every balance anywhere is a SQL view over postings; nothing is stored twice. Money is `numeric`, never floating point; time is `timestamptz`, shown in SYSTEM_SETTING.operating_timezone and in each partner's cutoff_timezone where the partner's clock applies.
Files	Amazon S3 for slips, screenshots and photos.	The model never stores a file or a bank detail in a table: it stores a reference. Objects go into one versioned bucket, private, encrypted at rest, reached through short-lived signed links the app issues; bank details, KYC and legal documents sit under a restricted prefix only Finance and Management roles can resolve.
Sign-in	NPLify's own login: email and password plus a code from an authenticator app. Users and roles are managed in NPLify.	USER (email, role_id, status) → ROLE (sees_economics) → ROLE_RIGHT (may_initiate, may_approve, escalates_to_role_id). Password hashed with a modern algorithm; TOTP secret per user; a session cookie, server-side, that expires on inactivity; every user change approved by a second Management user (Roles & Visibility rule 7). No outside identity provider: the ERD's note "Google Workspace" on USER.email describes the address, not the login.
Hosting	AWS in Hong Kong: one small container service, a managed PostgreSQL database and an S3 bucket.	Region ap-east-1. One container service running the app (App Runner or ECS on Fargate — one small task is enough), a managed PostgreSQL instance (RDS, smallest burstable class, storage auto-growing, automated backups on), one S3 bucket with versioning. Secrets (database password, rate-feed and explorer keys, the Sheets service account) in the account's secrets store, never in the image or the repository. Outbound calls only to the four services of Figure 1.

3 · The insert-only ledger, enforced by the database

NPL's page says the database refuses updates and deletes on ledger and audit tables. In the model, ledger tables are append-only and corrections are reversing transactions, and that is enforced in three layers so

that no screen, no job and no mistake can get around it:

1. **Privileges.** The application's database role has INSERT and SELECT on `LEDGER_TRANSACTION`, `LEDGER_POSTING`, `LEDGER_ACCOUNT` and `AUDIT_LOG` and *no* UPDATE or DELETE. Nobody uses the owner role day to day; schema changes are migrations run by deployment under a separate role.
2. **Triggers.** A BEFORE UPDATE OR DELETE trigger on the same tables raises an error whatever the role, so even an operator connected as the owner cannot change a posting by hand. `LEDGER_ACCOUNT` allows one exception: a row's `status` may be changed to closed; its code, owner, holder, currency and purpose may not.
3. **Balance at commit.** A deferred constraint trigger on `LEDGER_POSTING` checks, when a transaction commits, that its postings sum to zero per currency and, separately, per ownership tag, that every `CL.PROJECT.<CCY>.PAYABLE` line carries a `receiver_group_id`, and that the transaction names a source record of the stated `source_type`. An unbalanced transaction never reaches the books.

Everything else in the model is a normal row with a lifecycle (a deal, a settlement, a conversion, an approval) or a versioned configuration row (a fee structure, a threshold, a template, a system setting: new rows with an effective date, never edits). For those, `AUDIT_LOG` holds every field change with old and new value and the acting user, written by the API in the same database transaction as the change, and `RECORD_LOCK` prevents two people editing one record at once. Reversal is the only correction: `LEDGER_TRANSACTION.reversal_of_id` names the transaction reversed, and the Adjustment source is a Finance-initiated, Management-approved action (Lifecycle & Approvals §5).

4 · Background jobs the model needs

NPL's page names quote clocks, rate fetch and alerts. The model's clocks and checks need the nine jobs below; all run inside the single app process on a schedule table, each run recorded with its outcome so the exception list can show a job that failed. None of them moves money: a job may stamp a clock, expire a quote, attach evidence or raise an alert, but every posting is made by a controlled action of a person (or by the engine as part of one).

Job	When	What it does	Tables
Rate capture	every 60 s per active pair	Walks each CURRENCY_PAIR's PAIR_RATE_SOURCE chain (project pairs and platform pairs alike): Kraken first, CoinMarketCap next; writes a MARKET_RATE row per source that answered; marks a source stale when its snapshot is older than RATE_SOURCE.staleness_threshold_s. With every source down the job raises an alert and pricing waits for a manual rate entered by Finance with evidence and approved; it is recorded like any other market rate, with its approval.	`MARKET_RATE`, `RATE_SOURCE`, `PAIR_RATE_SOURCE`, `CURRENCY_PAIR`
Quote clocks	every 30 s	Moves quote-first legs along the clocks of the project: freshness (re-price before sending), validity (effective_valid_until, extended by QUOTE_VALIDITY_EXTENSION rows), grace (grace_until with the drift test); expires what passed; a late sender gets a new deal group.	`DEAL`, `DEAL_GROUP`, `QUOTE_PACKAGE`, `QUOTE_VALIDITY_EXTENSION`, `PROJECT`
Partner clocks and business days	every minute, in each partner's cutoff_timezone	Applies the effective cutoff (one-day override, else weekday rule, else PARTNER.bank_cutoff_time) to legs on pairs with needs_bank_lock: stamps cutoff_at and conversion_due_date, defers a next-day conversion that would breach the minimum margin, moves a settlement approved after cutoff to the next business day (execution_due_date) skipping weekends and HOLIDAY rows. Expires PARTNER_RATE_VERSION rows at expires_at and freezes a breached partner × pair until Finance decides.	`PARTNER`, `PARTNER_CUTOFF_OVERRIDE`, `HOLIDAY`, `PARTNER_PAIR`, `DEAL`, `SETTLEMENT`, `PARTNER_RATE_VERSION`, `RATE_COMPARISON`
Hash checks	on demand and every 5 min for pending ones	For a crypto collection or payout whose hash was shared: asks Tronscan (TRC-20) or Etherscan (ERC-20) for the transaction, compares address, amount and confirmations, and attaches the result as evidence. If the explorer does not answer, the item stays pending and Operations verifies by the explorer link; the job never auto-verifies a collection — verification is an Operations action with the evidence attached.	`COLLECTION`, `SETTLEMENT_LINE`, `CONFIRMATION`, `COLLECTION_RECEIVING_ENDPOINT`, `RECEIVING_ENTITY_ACCOUNT`

Job	When	What it does	Tables
Aging and exceptions	every 15 min	Builds the exception list every role sees in its own terms (Roles & Visibility §5, rule 8): custody beyond OWN_WALLET.custody_max_hours, open OFFSET rows past their aging, a pool below zero outside the allowed window, unfunded make-goods, open loss recoveries, frozen pairs, variance exceptions, destination approvals pending, settlements due today, single-use endpoints past their deal. Sends alerts to the role that must act.	`CUSTODY, OFFSET, PROJECT_BALANCE, LOSS_EVENT, LOSS_RECOVERY, DESTINATION_APPROVAL, SETTLEMENT, APPROVAL`
Exposure and reporting values	at request time, cached 60 s	REPORTING_VALUE is computed when a screen or report asks for it — never stored: the latest MARKET_RATE of the platform pair, refused as stale beyond reporting_rate_max_age_min. The same job compares Σ balances at partners with PROJECT.exposure_ceiling and CLIENT.exposure_ceiling and blocks a release that would cross either.	`REPORTING_VALUE, SYSTEM_SETTING, CLIENT_POSITION, PROJECT_BALANCE`
Monthly accruals and statements	first business day of the month, and on demand	Rolls PROJECT_MONTH_VOLUME for tier selection; produces the partner rebate statement to reconcile PARTNER_REBATE_ACCRUAL against the partner's statement and the markup-share statement per party; lists EARNINGS_RECEIVABLE rows resting at each partner for Finance to settle or offset.	`PROJECT_MONTH_VOLUME, PARTNER_REBATE_ACCRUAL, MARKUP_SHARE_ACCRUAL, EARNINGS_RECEIVABLE, EARNINGS_OFFSET`
Client journal	on every deal-state change, retried until confirmed	Appends one row to the client's Google Sheet for each state change of a deal group (quoted, sent, collected, converted, settled, confirmed, returned, voided) with sender-facing fields only (Roles & Visibility rule 2), then reads the row back and records the confirmation. A failed write stays queued and is shown on the exception list; the deal does not wait for the sheet.	`DEAL_GROUP, DEAL, SETTLEMENT_LINE, MESSAGE_TEMPLATE`
Backup verification	daily, after the backup window	Confirms the day's automated snapshot exists and restores last week's snapshot into a scratch database once a week to prove the restore path; checks S3 versioning is on. Reports to Management monthly.	`all tables`

5 · Screens, API and the visibility rule

There is one application with three views of it, by role. The **Operations** screens are the working day: inquiries and quotes, the quote package, wallet screening, collections to verify, conversions to instruct, settlements to prepare, destination approvals, loss events to record, the client journal; the **Finance** screens add the approvals queue, the partner rate versions and the day's comparison, variance and other exceptions, reconciliation, earnings resting at partners and offsets, invoices and the reseller books, statements, the Finance Log; the **Management** screens add the escalations queue, configuration proposals to approve, system settings, users and roles, and the reports in the reporting currency. Every screen is served by the same API, and the API applies the rule of the Roles & Visibility Matrix at the point where a response is built: for a role whose `ROLE.sees_economics` is false the fields of the groups *Reference rate and comparison*, *Margin, pool, cost breakdown* and *Ledger and balances* are removed before the

data leaves the server, on every read, list, export, document and notification. The browser never receives what the role may not see, so there is nothing to hide in the page.

Three API conventions carry the state machines: every state change is a named action (`POST /deals/{id}/actions/instruct-conversion`), never a field edit; an action that needs approval creates the APPROVAL row and returns the state the record is now in; the approver's call is a second action by a different user, checked against ROLE_RIGHT. Idempotency keys on every money-moving action, so a retried request cannot post twice.

6 · Environments, backups and running cost

Environment	Where	Data
Local	Developers' machines	Test data only; the configuration example of the Configuration Schema (config-example-client1.json) and the worked transactions of the Ledger Posting Design as fixtures.
Staging (temporary)	NPL's AWS account: a second small service and database, same image, own bucket	The parallel run of the Cutover Approach: NPL's real configuration, the backfilled deals and every deal mirrored from the spreadsheets for three to four weeks. Its database becomes production's at cutover; the instance is removed afterwards.
Production	NPL's AWS account	Phase test runs before go-live (NPL's page); reset once, just before cutover, and loaded from the staging database so the live ledger starts from reconciled opening entries and the mirrored days, not from test runs.

NPL's page had two environments and a single reset of production before go-live. The cutover plan (a staging instance for the parallel run, then a copy of its database to production) keeps the single reset and adds the temporary staging instance.

- **Backups.** Daily automated snapshots of the database kept 30 days, with point-in-time restore to any minute in that window (RDS automated backups); S3 versioning for files. A weekly restore test (Section 4, last job). A restore is a Management decision recorded in the audit log, because the ledger after a restore is the ledger as of that minute and the sheets fill the gap.
- **Monitoring.** The app's own exception list is the first monitor; underneath it, the account's basic alarms: service unhealthy, database storage or connections, a job run that failed twice, a rate feed stale for more than ten minutes, a journal row unconfirmed for more than an hour. Alerts go to Finance and the engineering contact.
- **Deployment.** One container image per release, built from the repository, migrations applied on start under the migration role, a release note that names the ERD edition the schema matches (the `version` of `erd.v5.json`, 5.9 today).

Running cost (NPL's estimate: about USD 50–80 a month)	Basis
Container service, one small task	USD 25–40: a 0.5 vCPU / 1 GB task running continuously, which also runs the jobs
Managed PostgreSQL, smallest burstable instance	USD 15–25 with 20 GB of storage and 30-day backups; the data is a few thousand rows a month
S3, secrets, logs, data transfer	USD 3–10
Staging instance during the parallel run	About the same again for the weeks it exists; removed at go-live

Rate-feed and explorer calls stay within the free tiers at this volume; Google Sheets is free; the authenticator app is the user's own. No licence is needed anywhere in the stack.

7 · Decisions for NPL and assumptions made here

1. **Container service.** App Runner (simplest) or ECS on Fargate (more control); both fit the cost line. Assumed: App Runner.

2. **Wallet screening provider**: which tool, and whether the app calls it or Operations pastes the result. Assumed: pasted result with evidence until a provider is chosen.
3. **The restricted store** for bank details, KYC and legal documents: the same bucket under a restricted prefix (assumed), or a separate secrets store.
4. **Client journal content**: which state changes and which fields each client sees; assumed the sender-facing set of MESSAGE_TEMPLATE quote plus state and timestamps, per client.
5. **Availability.** A single task and a single-AZ database are assumed; a short outage falls back to the sheets. Multi-AZ roughly doubles the database line and is not proposed at this scale.

This note describes the build; the data model it serves is the ERD & Data Model, and the rules it enforces are those of the Lifecycle & Approvals, Roles & Visibility, Calculation and Configuration documents.